



Python Operators

#PythonNotes

Operators in Python

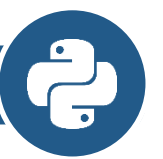


Operators in Python

- An operator is simply a symbol that is used to perform operations.
- There are following types of operators to perform different types of operations in Python...

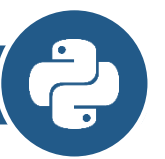
List of Operators

1. Arithmetic Operators
2. Comparison operators
3. Logical operators
4. Assignment operators
5. Membership Operators
6. Identity Operators



Arithmetic Operators

Operator	Meaning
+	Add two operands or unary plus
-	Subtract right operand from the left or unary minus
*	Multiply two operands
/	Divide left operand by the right one (always results into float)
%	Modulus - remainder of the division of left operand by the right
//	Floor division - division that results into whole number adjusted to the left in the number line
**	Exponent - left operand raised to the power of right(x to the power y)



Example

```
x = 10
y = 5

# addition
print('Sum: ', x + y)

# subtraction
print('Subtraction: ', x - y)

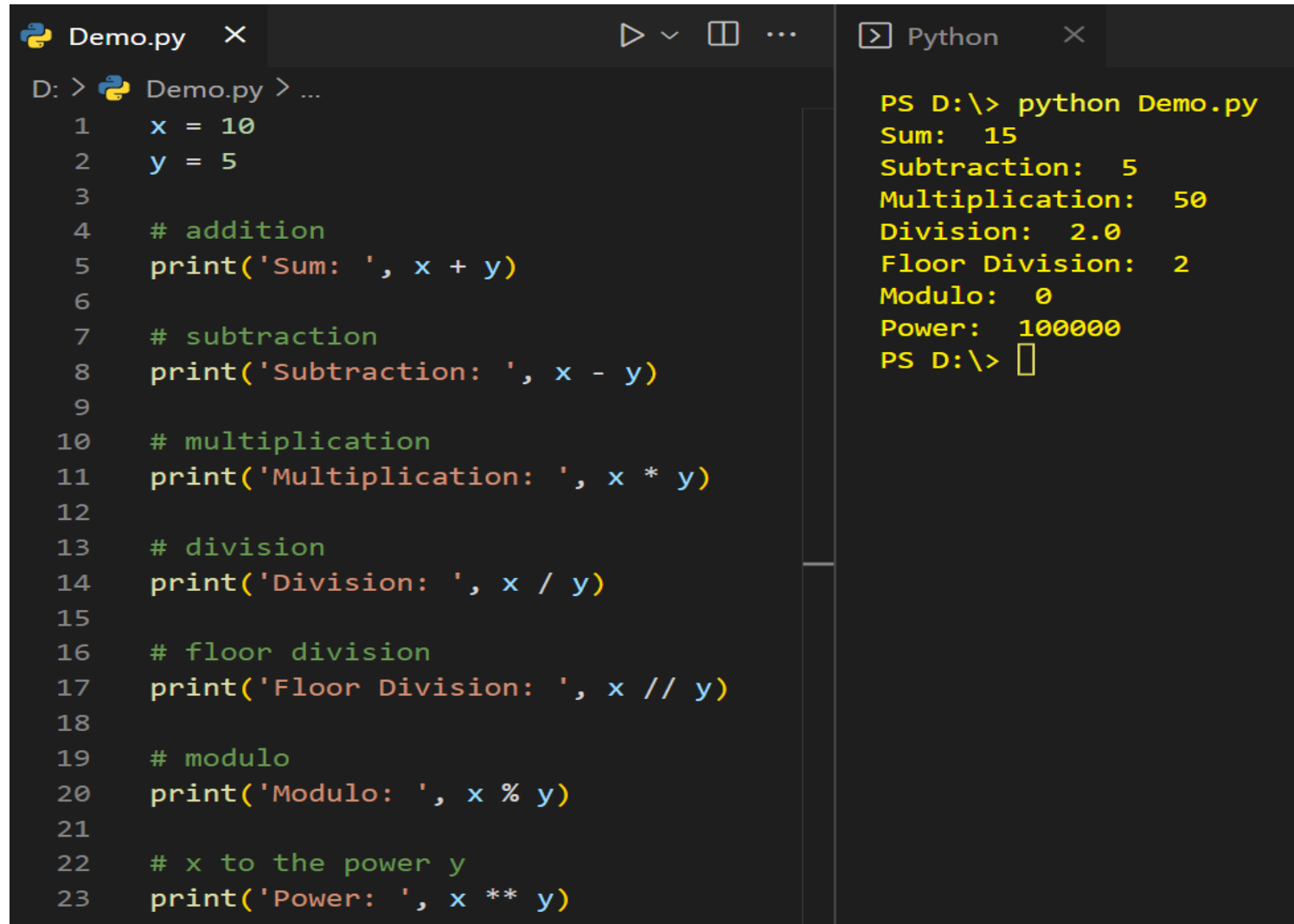
# multiplication
print('Multiplication: ', x * y)

# division
print('Division: ', x / y)

# floor division
print('Floor Division: ', x // y)

# modulo
print('Modulo: ', x % y)

# x to the power y
print('Power: ', x ** y)
```

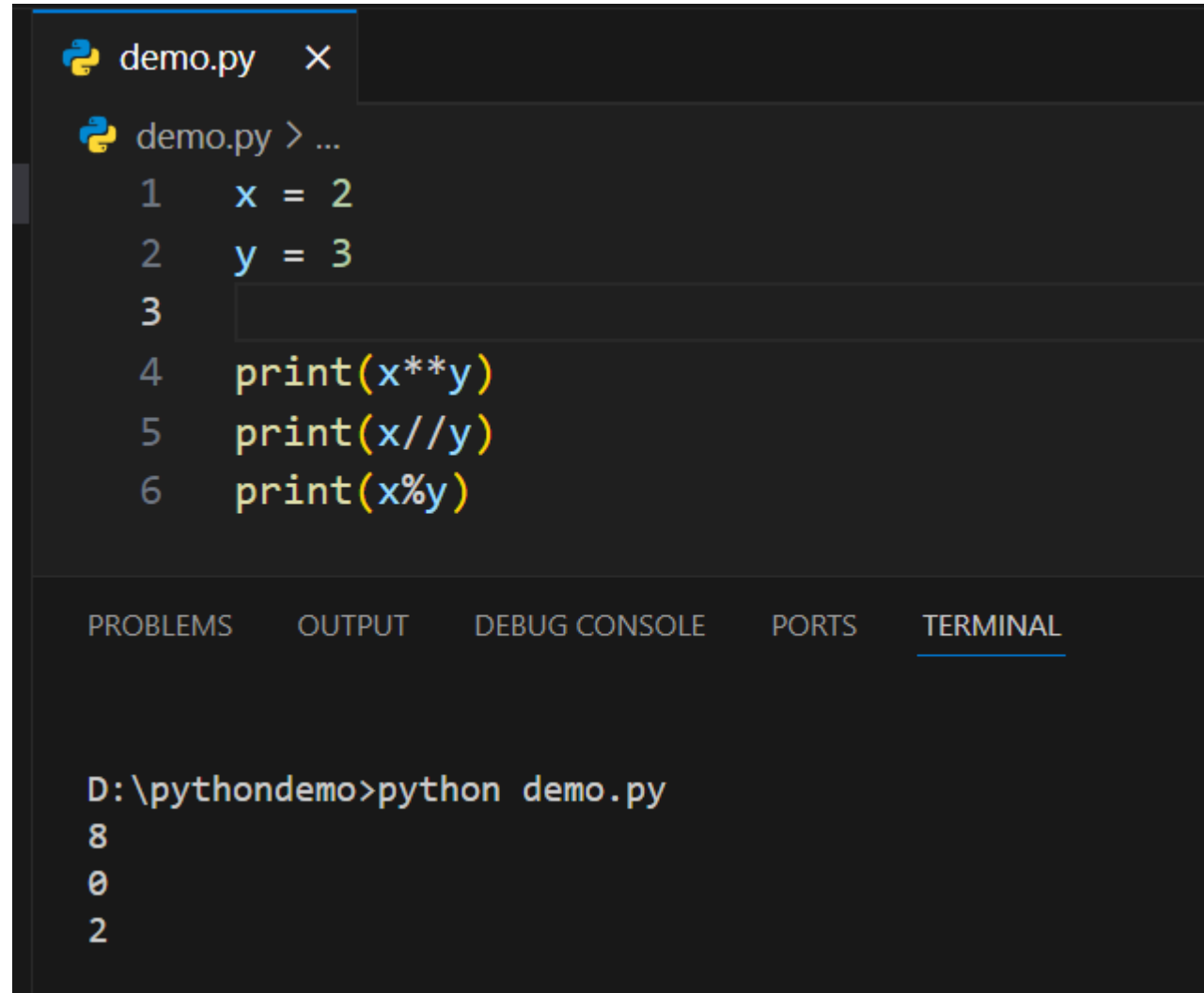


```
Demo.py ×
D: > Demo.py > ...
1 x = 10
2 y = 5
3
4 # addition
5 print('Sum: ', x + y)
6
7 # subtraction
8 print('Subtraction: ', x - y)
9
10 # multiplication
11 print('Multiplication: ', x * y)
12
13 # division
14 print('Division: ', x / y)
15
16 # floor division
17 print('Floor Division: ', x // y)
18
19 # modulo
20 print('Modulo: ', x % y)
21
22 # x to the power y
23 print('Power: ', x ** y)
```

```
PS D:\> python Demo.py
Sum: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
Floor Division: 2
Modulo: 0
Power: 100000
PS D:\>
```

Example

- $2^{**}3 = 2*2*2$
- $2//3 = 0$
- $2/3 = 2$



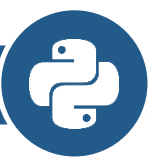
```
demo.py x
demo.py > ...
1 x = 2
2 y = 3
3
4 print(x**y)
5 print(x//y)
6 print(x/y)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\pythondemo>python demo.py
8
0
2
```

Comparison operators

Operator	Meaning	Example
>	Greater than	<code>x > y</code>
<	Less than	<code>x < y</code>
==	Equal to	<code>x == y</code>
!=	Not equal to	<code>x != y</code>
>=	Greater than or equal to	<code>x >= y</code>
<=	Less than or equal to	<code>x <= y</code>



Example

```
a = 10
b = 7
```

```
# equal to operator
print('a == b ', a == b)
```

```
# not equal to operator
print('a != b ', a != b)
```

```
# greater than operator
print('a > b ', a > b)
```

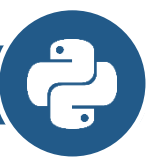
```
# less than operator
print('a < b ', a < b)
```

```
# greater than or equal to operator
print('a >= b ', a >= b)
```

```
# less than or equal to operator
print('a <= b ', a <= b)
```

```
Demo.py  × ... Python  ×
D: > Demo.py > ...
1  a = 10
2  b = 7
3
4  # equal to operator
5  print('a == b ', a == b)
6
7  # not equal to operator
8  print('a != b ', a != b)
9
10 # greater than operator
11 print('a > b ', a > b)
12
13 # less than operator
14 print('a < b ', a < b)
15
16 # greater than or equal to operator
17 print('a >= b ', a >= b)
18
19 # less than or equal to operator
20 print('a <= b ', a <= b)

PS D:\> python Demo.py
a == b  False
a != b  True
a > b   True
a < b   False
a >= b  True
a <= b  False
PS D:\> 
```



Assignment operators

Operator	Example	Equivalent to
=	<code>x = y</code>	<code>x = 5</code>
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>x -= 5</code>	<code>x = x - 5</code>
<code>*=</code>	<code>x *= 5</code>	<code>x = x * 5</code>
<code>/=</code>	<code>x /= 5</code>	<code>x = x / 5</code>
<code>%=</code>	<code>x %= 5</code>	<code>x = x % 5</code>
<code>//=</code>	<code>x //= 5</code>	<code>x = x // 5</code>
<code>**=</code>	<code>x **= 5</code>	<code>x = x ** 5</code>

Example

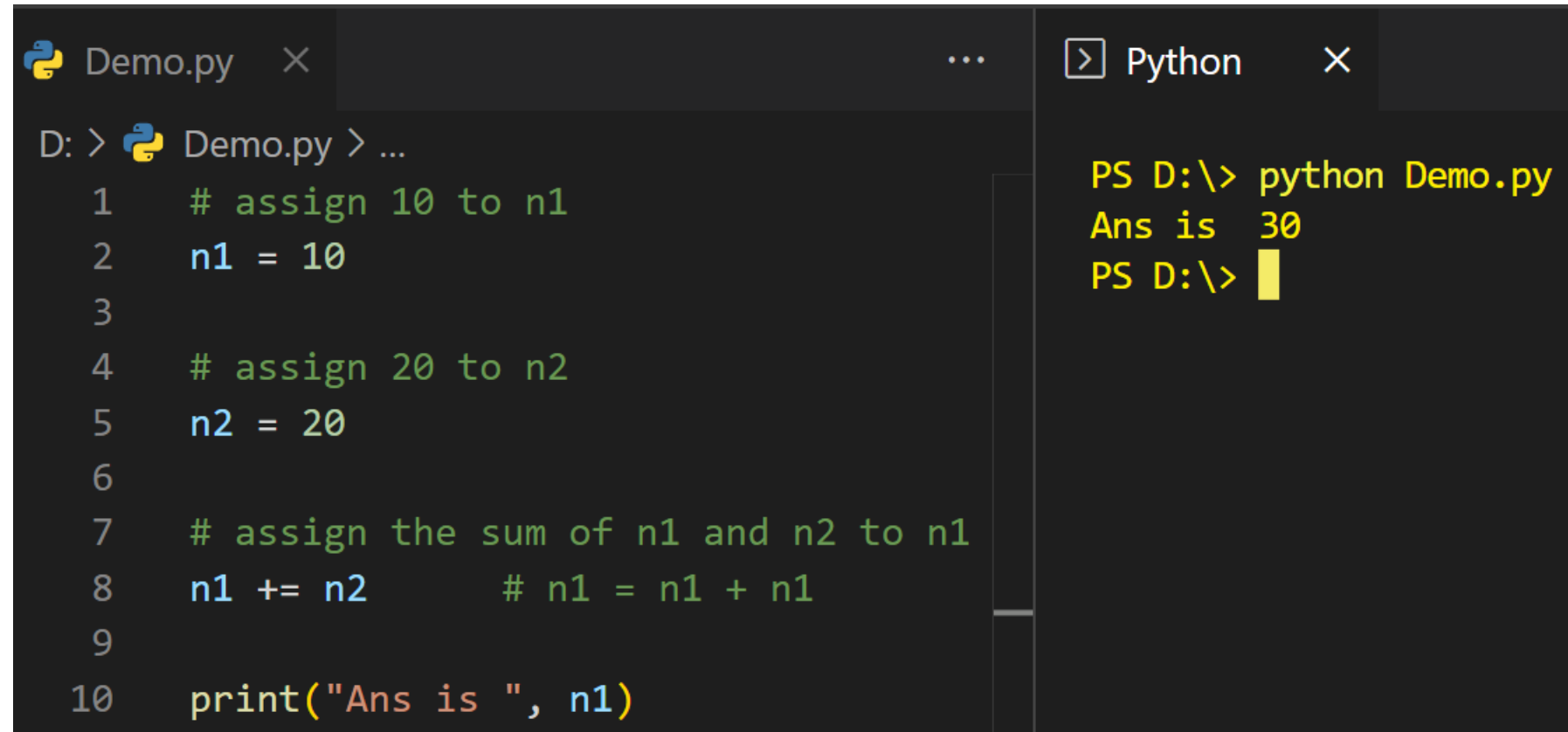
```
# assign 10 to n1
n1 = 10

# assign 20 to n2
n2 = 20

# assign the sum of n1 and n2 to n1
n1 += n2    # n1 = n1 + n2

print("Ans is ", n1)
```

```
1  n1 = 10
2  n2 = 20
3  temp = n1+n2
4  n1 = temp
5  print("Ans is",n1)
```



The screenshot shows a code editor window titled 'Demo.py' with the following code:

```
D: > python Demo.py > ...
1  # assign 10 to n1
2  n1 = 10
3
4  # assign 20 to n2
5  n2 = 20
6
7  # assign the sum of n1 and n2 to n1
8  n1 += n2    # n1 = n1 + n1
9
10 print("Ans is ", n1)
```

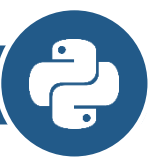
To the right, a terminal window titled 'Python' shows the command prompt output:

```
PS D:\> python Demo.py
Ans is 30
PS D:\>
```

```
demo.py ×
demo.py > ...
1  n1 = 10
2  n2 = 5
3
4  n2 += n1
5  print(n2)
6
7  n2 -= n1
8  print(n2)
9
10 n2 *= n1
11 print(n2)
12
13 n2 /= n1
14 print(n2)
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

```
D:\pythondemo>python demo.py
15
5
50
5.0
```



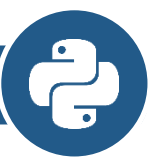
Logical operators

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

```
# logical AND  
print(True and True)    # True  
print(True and False)   # False
```

```
# logical OR  
print(True or False)    # True
```

```
# logical NOT  
print(not True)          # False
```

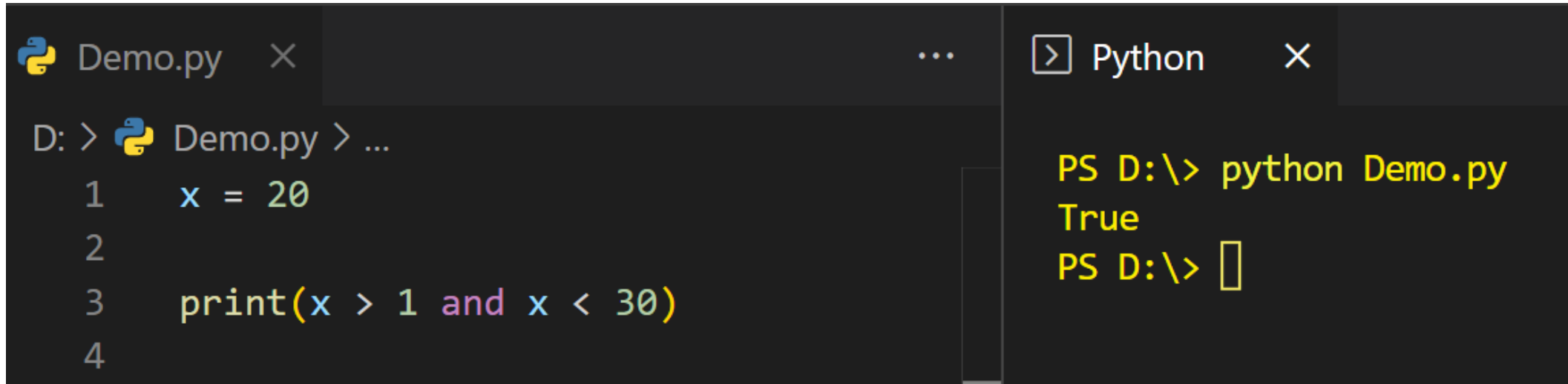


AND operator

- The logical AND operator checks whether all operands are true – the result is true only when all operands are true.

FIRST OPERAND	SECOND OPERAND	RESULT
true	true	true
true	false	false
false	true	false
false	false	false

Example of And Operator



```
Demo.py × ... Python ×  
D: > Demo.py > ...  
1 x = 20  
2  
3 print(x > 1 and x < 30)  
4  
PS D:\> python Demo.py  
True  
PS D:\> 
```

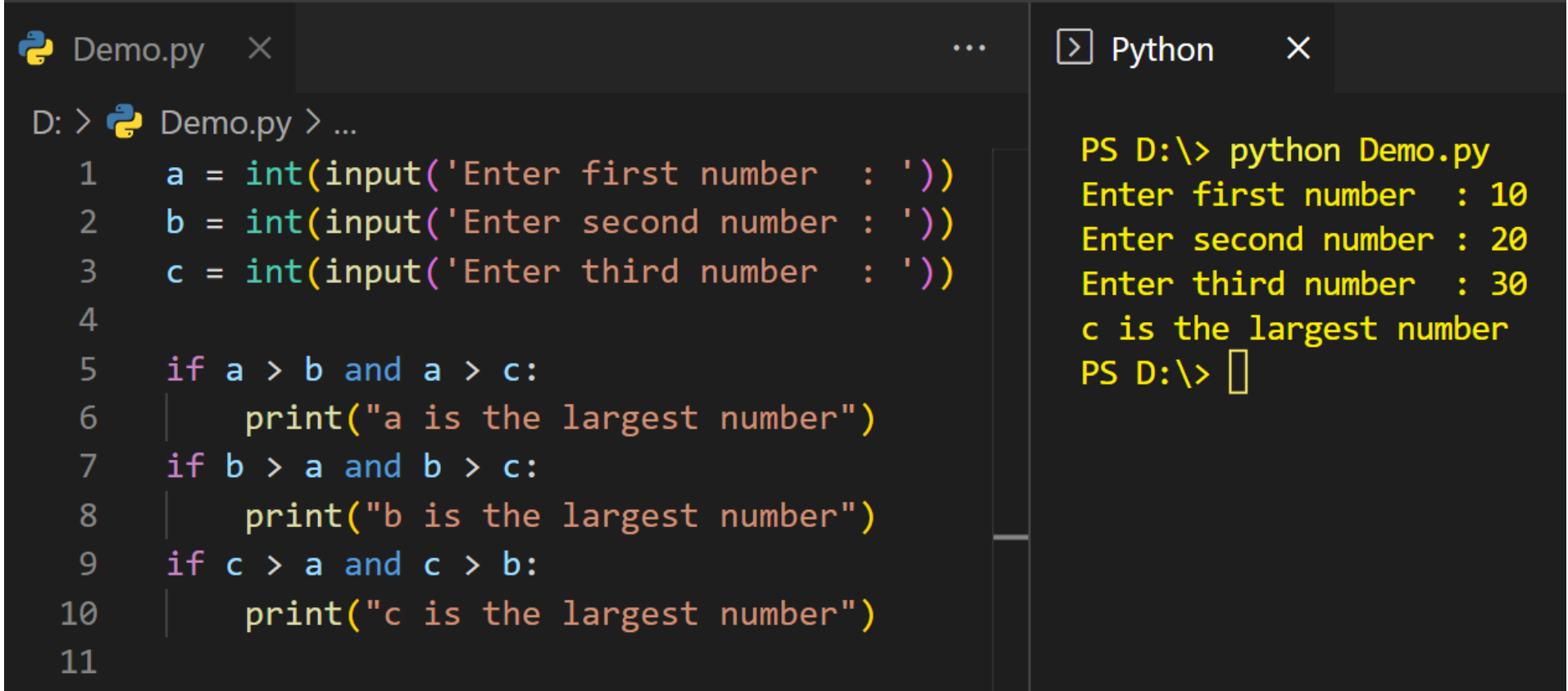
`x = 20`

`print(x > 1 and x < 30)`

Example2 – Find Maximum Number from 3 Number

```
a = int(input("Enter first number : "))
b = int(input("Enter second number : "))
c = int(input("Enter third number : "))
```

```
if a > b and a > c:
    print("a is the largest number")
if b > a and b > c:
    print("b is the largest number")
if c > a and c > b:
    print("c is the largest number")
```



```
Demo.py × ... Python ×

D: > Demo.py > ...

1  a = int(input('Enter first number : '))
2  b = int(input('Enter second number : '))
3  c = int(input('Enter third number : '))
4
5  if a > b and a > c:
6      print("a is the largest number")
7  if b > a and b > c:
8      print("b is the largest number")
9  if c > a and c > b:
10     print("c is the largest number")
11

PS D:\> python Demo.py
Enter first number : 10
Enter second number : 20
Enter third number : 30
c is the largest number
PS D:\> 
```

OR operator

- The logical OR operator checks whether one of the operands is true – the result is true if at least one of the operands is true.

FIRST OPERAND	SECOND OPERAND	RESULT
true	true	true
true	false	true
false	true	true
false	false	false

Example of Or Operator

```
Demo.py  X ... Python  X
D: > python Demo.py > ...
1     x = 20
2
3     print(x > 10 or x < 30)
4

PS D:\> python Demo.py
True
PS D:\> 
```

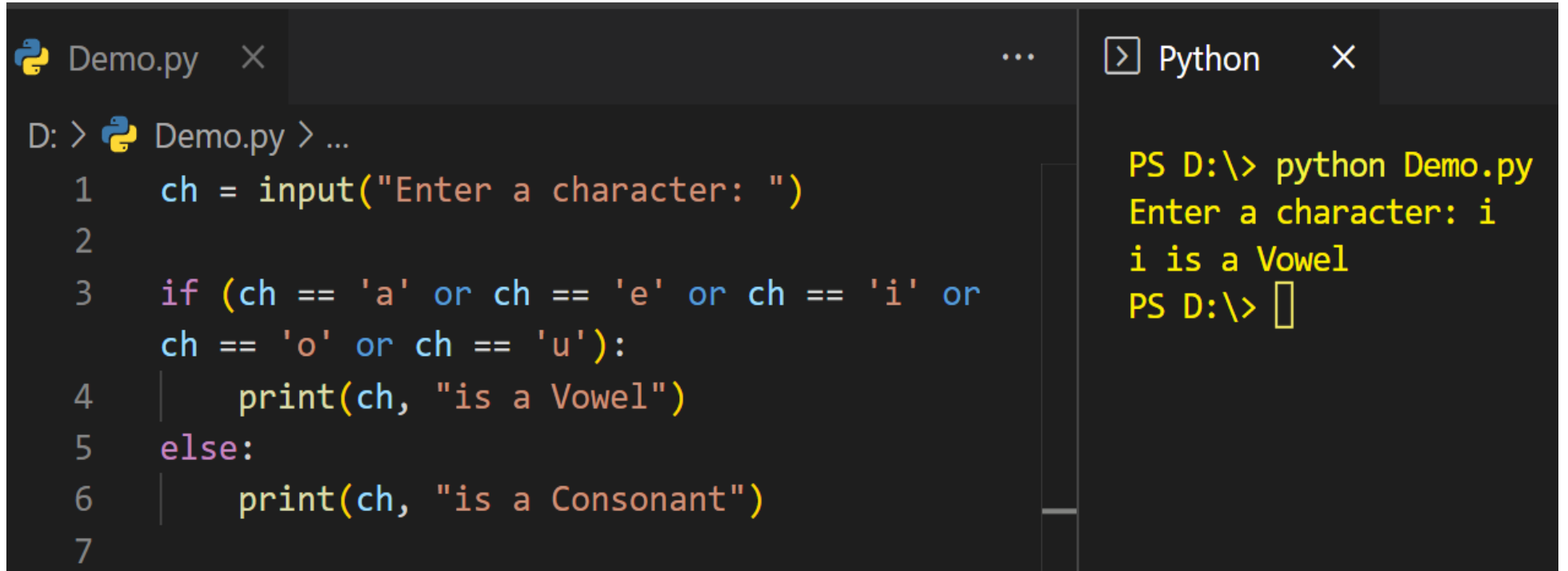
`x = 20`

`print(x > 10 or x < 30)`

Example2

```
ch = input("Enter a
character: ")

if (ch == 'a' or ch == 'e'
or ch == 'i' or ch == 'o'
or ch == 'u'):
    print(ch, "is a Vowel")
else:
    print(ch, "is a
Consonant")
```



The screenshot shows a code editor with two tabs: 'Demo.py' and 'Python'. The 'Demo.py' tab contains the following Python code:

```
D: > Demo.py > ...
1  ch = input("Enter a character: ")
2
3  if (ch == 'a' or ch == 'e' or ch == 'i' or
    ch == 'o' or ch == 'u'):
4      print(ch, "is a Vowel")
5  else:
6      print(ch, "is a Consonant")
7
```

The 'Python' tab shows the command prompt output:

```
PS D:\> python Demo.py
Enter a character: i
i is a Vowel
PS D:\> 
```

NOT operator

- The logical NOT operator returns the opposite of the operand.
- If the operand is true, it returns false.
- And if it is false, it returns true.

OPERAND	RESULT
true	false
false	true

Example of Not Operator



The screenshot shows a code editor window titled 'Demo.py' with the following code:

```
D: > Demo.py > ...  
1   x = 7  
2  
3   print(not(x > 1 and x < 10))  
4
```

To the right, a terminal window titled 'Python' shows the command prompt output:

```
PS D:\> python Demo.py  
False  
PS D:\> 
```

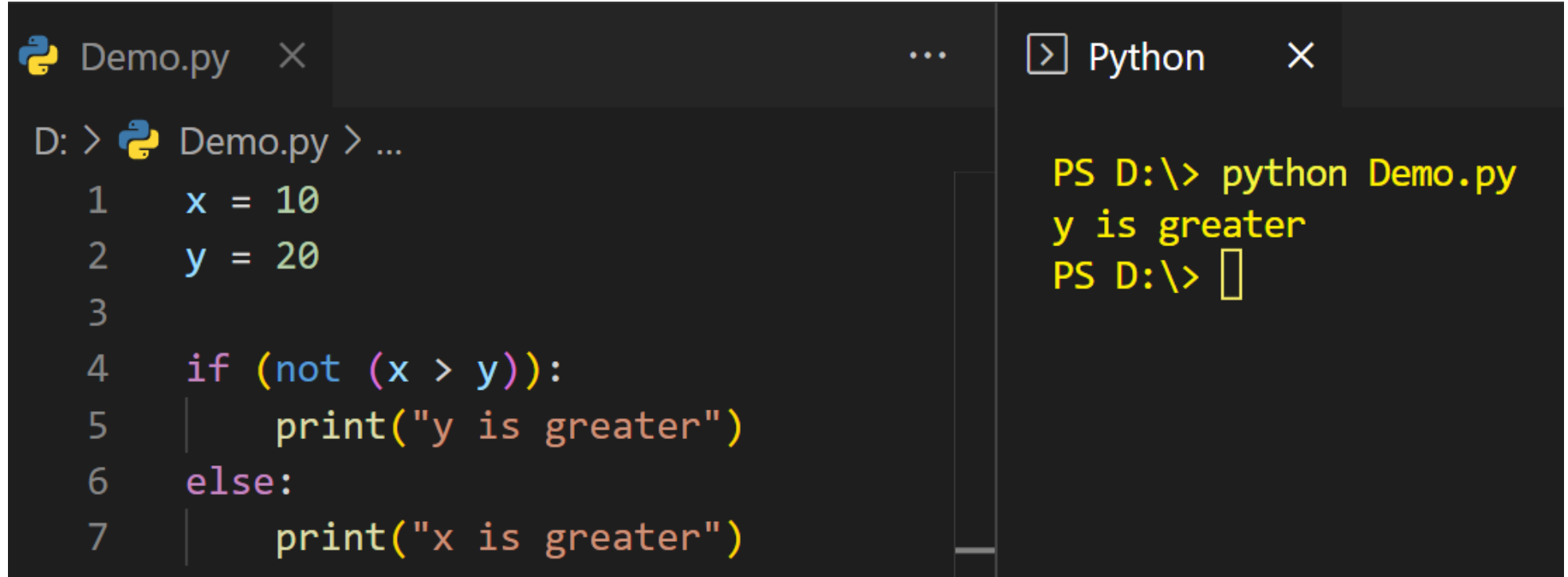
```
x = 7
```

```
print(not(x > 1 and x < 10))
```

Example2

```
x = 10
y = 20

if (not (x > y)):
    print("y is greater")
else:
    print("x is greater")
```



The screenshot shows a code editor with a file named 'Demo.py' and a terminal window. The code in the editor is as follows:

```
D: > Demo.py > ...
1 x = 10
2 y = 20
3
4 if (not (x > y)):
5     print("y is greater")
6 else:
7     print("x is greater")
```

The terminal window shows the command 'python Demo.py' being executed, resulting in the output 'y is greater'.

```
PS D:\> python Demo.py
y is greater
PS D:\>
```

Membership Operators

- There are two membership operators that are used in Python.
- They are used to test whether a value or variable is found in a sequence (string, list, tuple, and dictionary).

Operator	Meaning
in	True if value/variable is found in the sequence
not in	True if value/variable is not found in the sequence

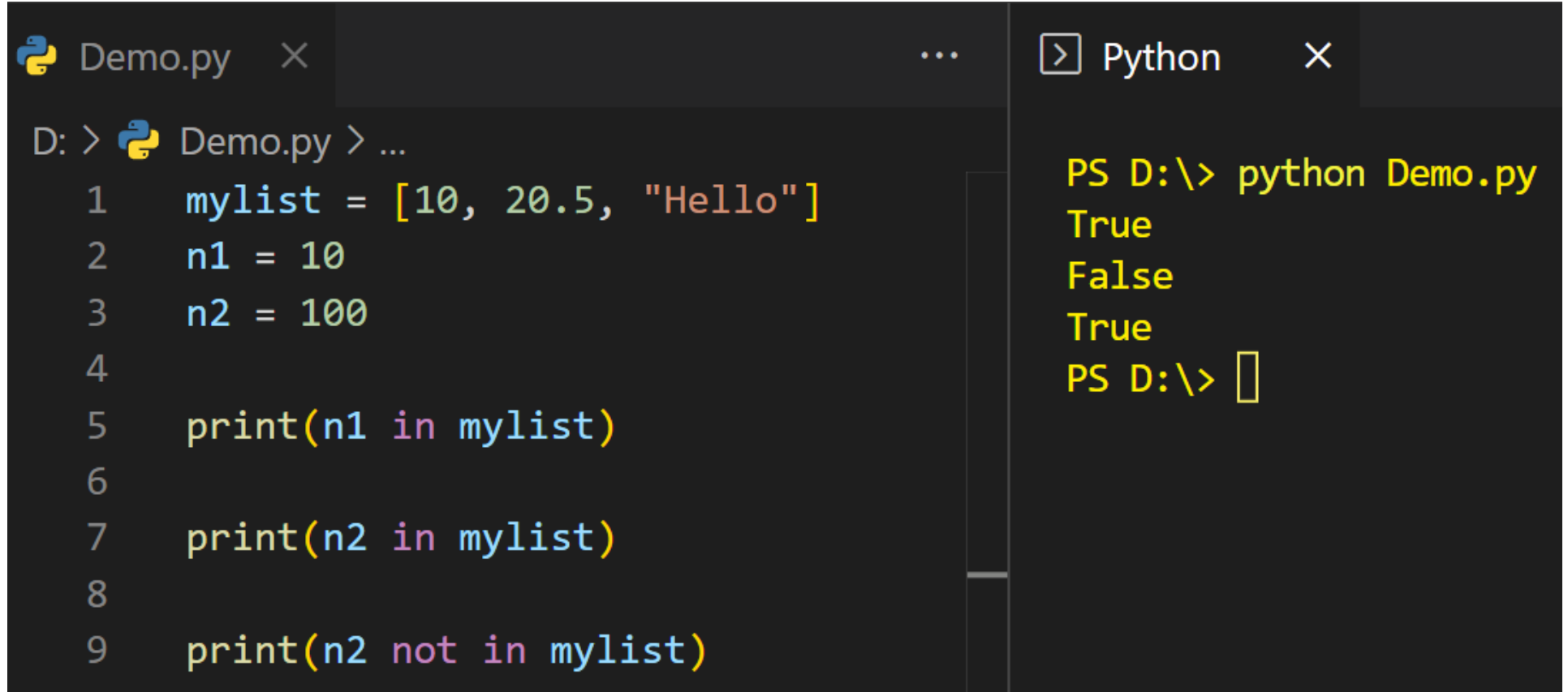
Example

```
mylist = [10, 20.5, "Hello"]  
n1 = 10  
n2 = 100
```

```
print(n1 in mylist)
```

```
print(n2 in mylist)
```

```
print(n2 not in mylist)
```



The screenshot shows a Python IDE with two windows. The left window, titled 'Demo.py', contains the following code:

```
D: > Demo.py > ...  
1 mylist = [10, 20.5, "Hello"]  
2 n1 = 10  
3 n2 = 100  
4  
5 print(n1 in mylist)  
6  
7 print(n2 in mylist)  
8  
9 print(n2 not in mylist)
```

The right window, titled 'Python', shows the output of running the script:

```
PS D:\> python Demo.py  
True  
False  
True  
PS D:\> 
```

Identity Operators

- To compare the memory location of two objects, Identity Operators are used. The two identify operators used in Python.

Operator	Meaning
is	True if the operands are identical (refer to the same object)
is not	True if the operands are not identical (do not refer to the same object)

Example

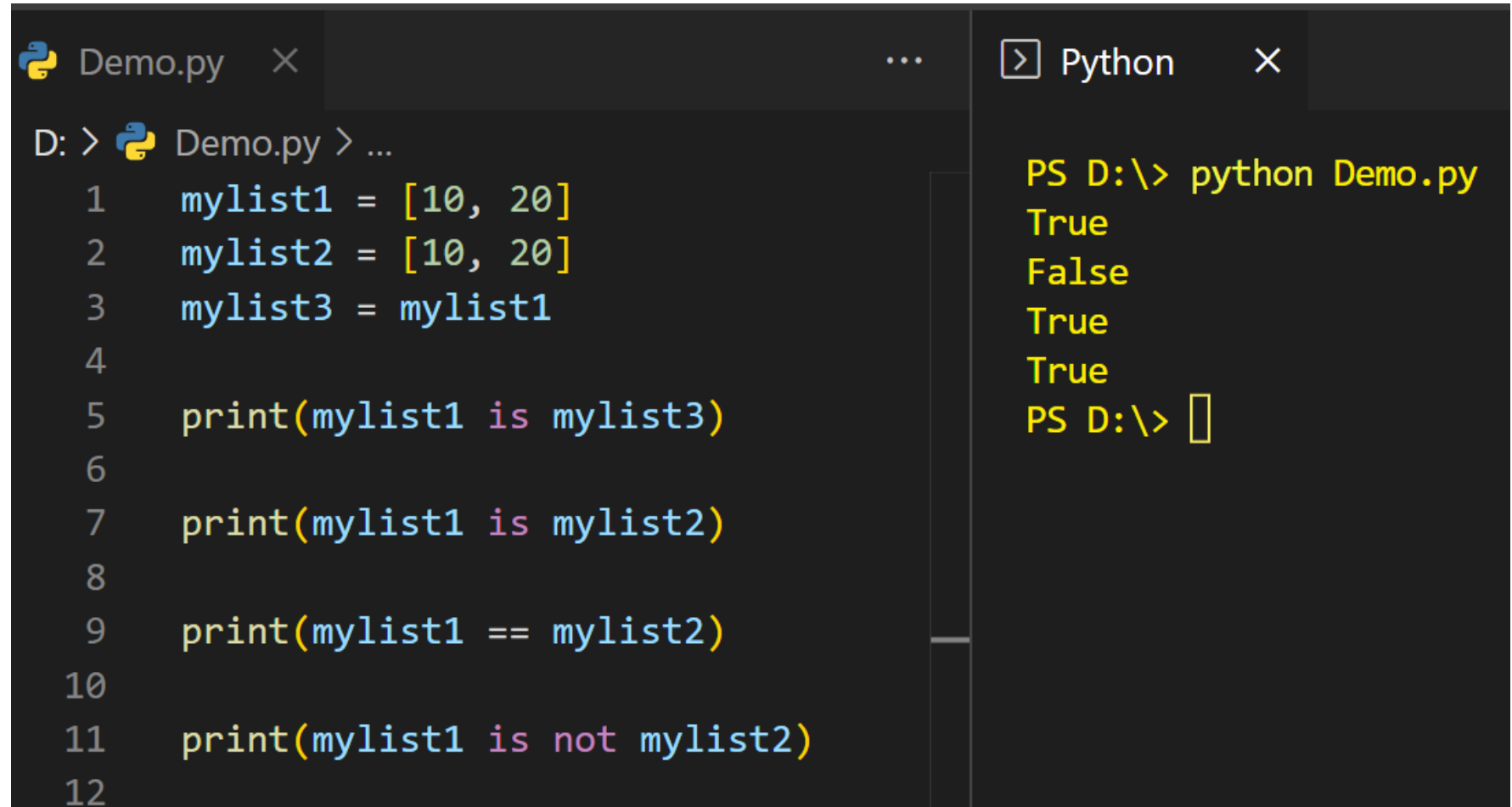
```
mylist1 = [10, 20]
mylist2 = [10, 20]
mylist3 = mylist1

print(mylist1 is mylist3)

print(mylist1 is mylist2)

print(mylist1 == mylist2)

print(mylist1 is not mylist2)
```



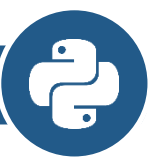
The screenshot shows a Python IDE with two windows. The left window, titled 'Demo.py', contains the following code:

```
D: > Demo.py > ...
1  mylist1 = [10, 20]
2  mylist2 = [10, 20]
3  mylist3 = mylist1
4
5  print(mylist1 is mylist3)
6
7  print(mylist1 is mylist2)
8
9  print(mylist1 == mylist2)
10
11 print(mylist1 is not mylist2)
12
```

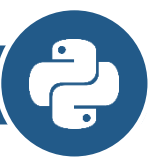
The right window, titled 'Python', shows the output of running the code:

```
PS D:\> python Demo.py
True
False
True
True
PS D:\> 
```

- # bitwise - 8421
- # a=10 # 1010
- # b=5 # 0101
- # print(a&b) # 0000-0
- # print(a|b) # 1111-15
- # print(a^b) # 1111-15
- # print(~a)
- # print(~b)
- # print(a<<b)# 1010 00000 - $2^6+2^8=320$
- # print(a>>b)# 00000 1010 - 0000 - 0



- # in, not in
- # s = "hello Python"
- # print('h' in s)
- # print('a' in s)
- # print('P' not in s)
- # print('b' not in s)



Let's Connect



Akash Padhiyar

Tech Educator • Consultant Tech
Mentor • Founder

On a Mission 🚀 to Bridge the
Gap Between Campus & Corporate

Akashsir.com

SOCIAL MEDIA

#DailyUpdates



@akashsirdotcom

#FreeCode



@akashpadhiyar

#TechUpdates



@akashpadhiyar

#FreeTutorials



@akashtecholabs

#HelpDesk



+91 99786 21654

WEBSITES

> **www.akashtecholabs.com**

> **www.akashsir.com**



akash@akashtecholabs.com

📍 202, Arya Arcade, Mithakhali Six Road, Navrangpura, Ahmedabad, Gujarat 380009